



TRUST404 · TRACK 04

AutoPoC

Autonomous Exploit-Proof Agent

취약점을 "**의심**"으로 보고서에 남기지 않습니다.

공격이 실제로 성립함을 **실행되는 PoC로 스스로 증명**하는 에이전트입니다.

실행 증명

캐시 결정론

오탐 0

스스로 도출

미공개 일반화

01 문제 정의

- ◆ 감사의 병목은 "확인"에 있습니다. 정적 분석·LLM 도구는 취약 가능성을 쏟아내지만, 실제 성립 여부를 판단하는 일은 여전히 사람의 몫이며 느리고 비용이 큼니다.
- ◆ "가능성 리포트"는 신뢰하기 어렵습니다. 오탐(false positive)이 섞여 있어, 리포트 한 줄마다 사람이 다시 검증해야 합니다.
- ◆ 재현이 어렵습니다. 같은 입력에도 결과가 흔들리면 회귀 검증과 감사 추적이 불가능합니다.
- ◆ → 그래서 필요한 것은, 공격 성립을 스스로 · 재현 가능하게 · 오탐 없이 증명하는 자율 에이전트입니다.

02 위협 모델

입력 & 공격자 능력

타깃 컨트랙트 소스와 불변식(*Invariants*), manifest를 입력으로 받습니다. 공격자는 임의의 트랜잭션 시퀀스와 재진입, 외부 콜, 자금 이동을 수행할 수 있습니다.

신뢰 경계

LLM은 오직 소스만 보고 추론합니다. 파일·웹 접근이 없고 타깃별 정답 하드코딩도 없으며, 모든 후보는 하네스 실행을 통해서만 채택됩니다.

성공의 정의

조직위 하네스의 `!!invariants.checkAll`이 실제로 깨질 때에만 성공으로 인정됩니다. 에이전트도 LLM도 판정 권한을 갖지 않으며, 판정은 오직 forge 실행이 내립니다.

안전장치

정상 컨트랙트에는 거짓 성공을 만들지 않고 **기권합니다(exit 1)**. 네트워크는 허용된 LLM API 예외만 사용하며, 오프라인에서는 완전 차단 상태로 동작합니다.

입력 · 타깃 소스 · **Invariants.sol** · manifest.json



TRIAGE — 정적 분류로 취약 유형(vuln_class) 판별 수 ms



ROUTER — 유형별 두뇌 우선순위 결정 오프라인 결정론 두뇌를 클라우드보다 먼저



Fuzzer

Reentrancy

Cloud LLM

Heuristic

지식 카드(RAG-lite)가 LLM 프롬프트에 힌트를 주입하며, 가장 먼저 실행으로 증명된 후보가 채택됩니다

▼ 각 후보

VERIFICATION GATE — forge 하네스로 실제 실행 → **checkAll이 깨지는가?**

깨짐 → **PROVEN**

솔루션 캐시에 고정 → **Exploit.sol + attempts.log** ·
exit 0

안 깨짐 → 자가수정 ↻

실패를 피드백으로 다음 두뇌/재시도에 주입

예산 소진 → 기권

거짓 성공 없이 **exit 1**

Triage가 유형을 분류하면 그 유형에 **가장 강한 두뇌를 먼저** 실행합니다. 모든 두뇌는 동일한 실행 검증 게이트 뒤에 놓이므로, 약한 두뇌의 오답은 채택되지 않습니다.

두뇌	담당(강점) 유형	방식	오프라인	결정론
Fuzzer	접근제어 · 산술	invariant 퍼징 → 축약 반례 → PoC 합성 (사전지식 0)	✓	✓
Reentrancy	재진입	외부콜-선행 함수 탐지 → 재진입 액터 합성	✓	✓
Cloud LLM	오라클 · delegatecall · 신종	소스 추론 + 실패 자가수정 · 폴백 체인 sonnet-5→opus-4-8	키 필요	캐시로
Heuristic	tx.origin · delegatecall 등	정적 공격 템플릿(결정론 최후수단)	✓	✓

"그냥 AI"도 "그냥 퍼저"도 아닙니다. **각 유형의 최강 도구를 하나의 검증 게이트 뒤에 결합했습니다.** 재진입처럼 퍼저가 다루기 어려운 클래스는 전용 브레인이, 퍼즐형은 LLM이 담당합니다.

강점 01 증명되지 않으면 성공이 아닙니다

실행 · 오탐

■ 실행 증명 게이트

모든 후보는 **forge** 하네스에서 실제로 실행되며, **checkAll**을 실제로 깨뜨릴 때에만 채택됩니다. "될 것 같다"는 후보는 산출물이 되지 못합니다. 판정 권한은 하네스에만 있습니다.

■ 오탐 0 — 밀짖하면 기권

취약하지 않은 타깃에서는 예산을 모두 소진하더라도 **exit 1**로 기권합니다. 거짓 성공(false positive)을 구조적으로 만들지 않습니다.

제출되는 모든 성공은 이미 실행으로 증명된 결과입니다. 실행·재현 실패의 위험을 산출 단계에서 제거하고, 취약한 컨트랙트와 정상 컨트랙트를 **exit 0**과 **exit 1**로 명확히 구분합니다.

■ 솔루션 캐시

성공한 PoC를 (타겟 소스 해시, 시드)로 고정합니다. 재실행 시에는 LLM을 다시 호출하지 않고, **바이트 단위로 동일하게** 리플레이합니다.

■ LLM 비의존 결정론

퍼저 시드를 고정하고 오프라인 결정론 두뇌를 우선합니다. 덕분에 결정론이 **LLM 온도·샘플링에 흔들리지 않습니다**.

■ 검증됨

오프라인 **N=3회 신규 생성이 바이트 동일**하고, 클라우드는 **캐시 리플레이가 동일함**을 실측으로 확인했습니다.

동일한 조건에서는 **동일한 PoC**가 나옵니다. 생성 경로(캐시 없는 재생성)와 재현 경로(캐시 리플레이)가 모두 결정론적이므로, 회귀 검증과 감사 추적이 가능합니다.

스스로 도출

정답 하드코딩이 없습니다. 클라우드 브레인은 파일·웹 접근이 없어 전달된 소스만 참고하고, 퍼저는 사전지식 없이 반례를 탐색합니다. 공개된 익스플로잇을 "재현"하는 것이 아닙니다.

미공개 일반화

모델 폴백 체인(sonnet-5 → opus-4-8)이 처음 보는 유형을 추론으로 공략합니다. 실측에서 포팅한 미공개 유형 8건을 모두 발견했습니다.

최소 PoC

퍼저가 축약한 반례로부터, 호출하는 인터페이스만 선언한 `self-contained run(address)` PoC를 합성합니다.

에이전트가 소스로부터 직접 도출한 경로이며(암기·카피가 불가능합니다), 처음 보는 타깃으로도 일반화되고, 산출물은 재현 가능한 최소 PoC입니다.

6/6

공개셋
(취약 4 발견 + 멀쩡 2 기권)

8/8

포팅 미공개 유형
(클라우드 폴백)

14/14

종료코드 정확
(발견 0 / 기권 1)

100%

결정론
바이트 동일 재현

항목	결과	근거
공개셋 4종 취약점	전부 오프라인 발견	재진입·퍼저·휴리스틱, 네트워크 차단 하에서도 PROVEN
멀쩡한 타깃 2종	오탐 0 · 기권	예산 소진 후 exit 1, 거짓 성공 없음
미공개 유형 8종	전부 예산 내 발견	클라우드 추론 일반화 (파일·정답 접근 없음)
결정론	바이트 동일	오프라인 N=3 재생성 + 클라우드 캐시 리플레이
채점 조건	준수	Foundry v1.0.0 · 하네스 byte-identical · validate-submission PASS

06 왜 AutoPoC인가

■ 증명 기반

"의심"이 아니라 **실행되는 PoC**입니다. 하네스가 깨질 때에만 성공이므로, 사람이 다시 검증할 필요가 없습니다.

■ 캐시 결정론

같은 입력에는 **바이트 동일한** 결과가 나옵니다. LLM의 비결정성과 무관하게 재현과 감사가 가능합니다.

■ 오탐 0

정상 타깃에는 기권합니다. 리포트의 모든 성공이 **실제로 성립함**을 보장합니다.

■ 스스로 도출

파일·정답 접근 없이 소스에서 직접 유도합니다. 공개 익스플로잇의 카피가 아닙니다.

■ 미공개 일반화

폴백 체인으로 처음 보는 유형까지 공략하며, 포팅 8/8을 달성했습니다.

■ 최소 PoC

축약 반례에 기반한 self-contained `run(address)` 입니다.

실행되는 PoC가 곧 신뢰입니다. **AutoPoC는 취약점을 주장하지 않고 증명합니다.**